

SECURITY ASSESSMENT

Grand Marina Hotel Water Monitoring System

Prepared for: Marcus Webb, General Manager

Prepared by: Pracheth Godlaveti

Date: 6/3/26

EXECUTIVE SUMMARY

The Grand Marina is ready to deploy their HYDROLOGIC water monitoring system. We tested four scenarios in two weeks. We specifically checked if outsiders could spy on sensor data, if the system prevents unauthorized access, how much encryption slows down processes, and if the system holds up under emergency conditions. The results are clear. Without security enabled, anyone on the hotel network could read the sensor data and send remote commands to the system. These include emergency shutoffs, and they could do so with a single line of code and no credentials. However, with security enabled, those attack possibilities quickly diminish. The performance cost of implementing this is negligible, since we found that encryption adds less than a quarter of a millisecond of delay with each reading. Also, the system handled over 30 times the hotel's regular sensor load with no problems. Thus, we **strongly** recommend enabling TLS encryption on all three HYDROLOGIC devices immediately.

SECURITY RISKS ADDRESSED

Before Security:

Without encryption, anyone on the hotel network can read sensor data and issue commands to the system in two seconds with a single command. This includes triggering emergency shutoffs, and gaining the potential for replay and spoofing attacks to silence real leak alerts and create false readings. This is exactly how the attackers in the \$4.2M incident gained leverage.

After Security:

Unencrypted connections from random devices on the network are instantly rejected and cannot deliver or receive any data. Devices need to give credentials at certificate

verification steps, so they can't even get in. And even if they do, all sensor data is encrypted while it is transferred along the MQTT pathway.

PERFORMANCE IMPACT

What We Measured	Before Security	After Security	Verdict
How fast readings arrive (average)	50.39 ms	50.62 ms	Negligible difference
How fast readings arrive (min/max)	0.31 ms/100.66 ms	0.56 ms/100.61ms	Comparable
All sensors at once	100/sec	100/sec	Passes all load levels

Encryption increases the delivery time by 0.23 ms per reading. For reference, a blink takes 65x that long. At the level the Grand Marina devices operate at, this is practically speaking, undetectable.

```
Administrator: Windows PowerShell
CA Certificate: NONE
=====
SUCCESS: Connected to broker!
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04> python experiment_runner.py --mode 1
ancy --tls off --count 50
=====
Latency Test
TLS: OFF
Messages: 50
=====
Sent 10/50 messages...
Sent 20/50 messages...
Sent 30/50 messages...
Sent 40/50 messages...
Sent 50/50 messages...
=====
Latency Results (TLS OFF)
=====
Messages sent: 50
Average latency: 50.39 ms
Min latency: 0.31 ms
Max latency: 100.66 ms
Std deviation: 50.21 ms
=====
```

Figure 1: Latency test output with TLS off

```
Administrator: Windows PowerShell
Std deviation: 50.21 ms
=====
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04> ^C
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04> python experiment_runner.py --mode 1a
tency --tls on --count 50
=====
Latency Test
TLS: ON
Messages: 50
=====
Sent 10/50 messages...
Sent 20/50 messages...
Sent 30/50 messages...
Sent 40/50 messages...
Sent 50/50 messages...
=====
Latency Results (TLS ON)
=====
Messages sent: 50
Average latency: 50.62 ms
Min latency: 0.56 ms
Max latency: 100.61 ms
Std deviation: 50.02 ms
=====
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04>
```

Figure 2: Latency test output with TLS on

TESTING EVIDENCE

We ran four security tests on the water monitoring system:

1. Can outsiders see sensor data?

Without security, we captured all five test messages in plain text using a separate terminal without credentials. When we enabled security, we got an immediate error and nothing was delivered.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Windows\system32> cd "C:\Users\Pracheth.Godavelli\OneDrive\Documents\hydroefficient-project\project-04"

PS C:\Users\Pracheth.Godavelli\OneDrive\Documents\hydroefficient-project\project-04> mosquito_sub -- localhost -- 1883
"grandinia/m" ->
randmania/sensors/test/telemetry ("device_id": "test-sensor-001", "message_num": 1, "pressure_psi": 82.5, "flow_rate
m": 45.6, "timestamp": "20:42:18")
randmania/sensors/test/telemetry ("device_id": "test-sensor-001", "message_num": 2, "pressure_psi": 83.5, "flow_rate
m": 45.6, "timestamp": "20:42:19")
randmania/sensors/test/telemetry ("device_id": "test-sensor-001", "message_num": 3, "pressure_psi": 84.5, "flow_rate
m": 45.6, "timestamp": "20:42:20")
randmania/sensors/test/telemetry ("device_id": "test-sensor-001", "message_num": 4, "pressure_psi": 85.5, "flow_rate
m": 45.6, "timestamp": "20:42:21")
randmania/sensors/test/telemetry ("device_id": "test-sensor-001", "message_num": 5, "pressure_psi": 86.5, "flow_rate
m": 45.6, "timestamp": "20:42:22")
PS C:\Users\Pracheth.Godavelli\OneDrive\Documents\hydroefficient-project\project-04> mosquito_sub -- localhost -- 8883
"grandinia/m" ->
non: protocol error
PS C:\Users\Pracheth.Godavelli\OneDrive\Documents\hydroefficient-project\project-04> mosquito_sub -- localhost -- 8883
"grandinia/m" ->
non: protocol error
PS C:\Users\Pracheth.Godavelli\OneDrive\Documents\hydroefficient-project\project-04>
```

Figure 3: Plain text messages being delivered (security off) vs seeing a TLS protocol error (security on)

2. Does the system block unauthorized connections?

This was binary. With correct credentials, a successful connection occurred. Wrong credentials failed immediately citing a certificate verification failure. This access was all or nothing.

```
Administrator: Windows PowerShell
connect --tls on

=====
Connection Test
TLS: ON
CA Certificate: certs/ca.pem
=====

SUCCESS: Connected to broker!
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04> python experiment_runner.py --mode generate-wrong-ca
>>

Generating wrong CA certificate for testing...
Saved: certs/wrong-ca.pem
To test: Use this as ca_certs in your client
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04> python experiment_runner.py --mode test-wrong-ca
>>
Testing connection with wrong CA...

=====
Connection Test
TLS: ON
CA Certificate: certs/wrong-ca.pem
=====

FAILED: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self-signed certificate in certificate chain (_ssl.c:1081)
PS C:\Users\Pracheth Godlaveti\OneDrive\Documents\hydroficient-project\project-04>
```

Figure 4: FAILED result for wrong CA scenario

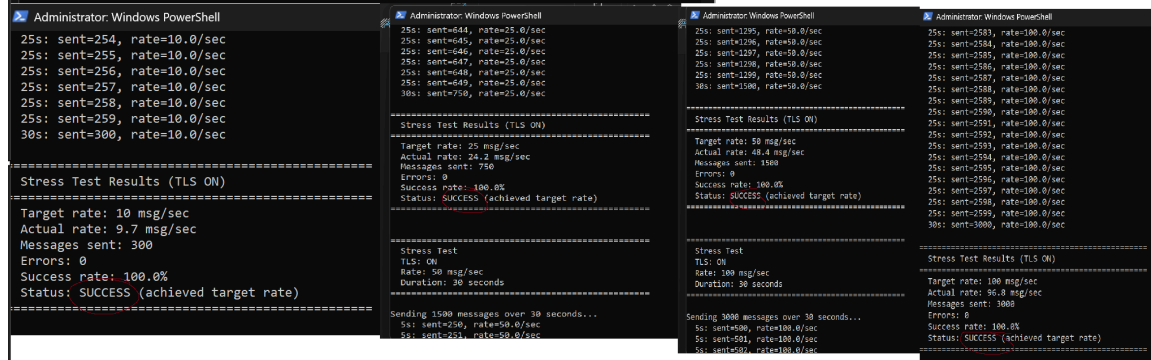
3. Does security slow things down?

We calculated that encryption added 0.46% overhead latency; 50.39ms without TLS vs 50.62ms with TLS. For a system that cares about water flow in intervals of five seconds, this is completely irrelevant.

4. Can the system handle a crisis?

The secured system passed all four load levels of 10, 25, 50, and 100 messages per second. In fact, the *unsecured* version actually degraded at 25 messages per second while the secured one did not.

```
PS C:\Users\Pracheth.Godaveti\OneDrive\Documents\hydroficient-project\project-04> python experiment_runner.py --mode stress --tls on --rate 10 --duration 30
>> python experiment_runner.py --mode stress --tls on --rate 25 --duration 30
>> python experiment_runner.py --mode stress --tls on --rate 50 --duration 30
>> python experiment_runner.py --mode stress --tls on --rate 100 --duration 30
```



Rate (msg/sec)	Target Rate	Actual Rate	Messages Sent	Errors	Success Rate	Status
10	10	9.7	300	0	100.0%	SUCCESS (achieved target rate)
25	25	24.2	750	0	100.0%	SUCCESS (achieved target rate)
50	50	48.4	1500	0	100.0%	SUCCESS (achieved target rate)
100	100	96.8	3000	0	100.0%	SUCCESS (achieved target rate)

Figure 5: Stress test output showing SUCCESS results for TLS ON across all rates

RECOMMENDATION

We would recommend **immediately** enabling TLS encryption on all three HYDROLOGIC devices. Without it, the system is literally one unsecured hotel network connection away from an attacker who literally dictates the water flow of the building and beyond. They can silence alerts and trigger shutoffs with no trace. The average hospitality data breach cost \$3.86 million in 2024, and 60% of hotel cyberattacks by 2025 are projected to come from IoT device vulnerabilities. We have a fix ready and tested, so there is no reason to wait on this.

NEXT STEPS

1. Switch all three HYDROLOGIC devices to the TLS configuration today without any changes to dashboard or alert behaviors.
2. Require an immediate certificate verification (mTLS) for any new device added to the system in the last week.
3. Set up alert escalation for unacknowledged alerts within a month. This way, any missed critical alert (whether it be the fault of the operator or the doing of an attacker) immediately and automatically notifies a backup contact within minutes.